

<best title wins>



Sebastian Kuehnau
Vaadin Enthusiast

vaadin}>

Vom **User Prompt** zur JPA-Query

Tool Calling, Structured Output &
ein hybrider Ansatz im Vergleich



Sebastian Kuehnau
Vaadin Enthusiast

Who's speaking?

Sebastian Kühnau

Developer Advocate at Vaadin

+20y in web- & java development

+10y at Vaadin

several technical roles

now developer advocate

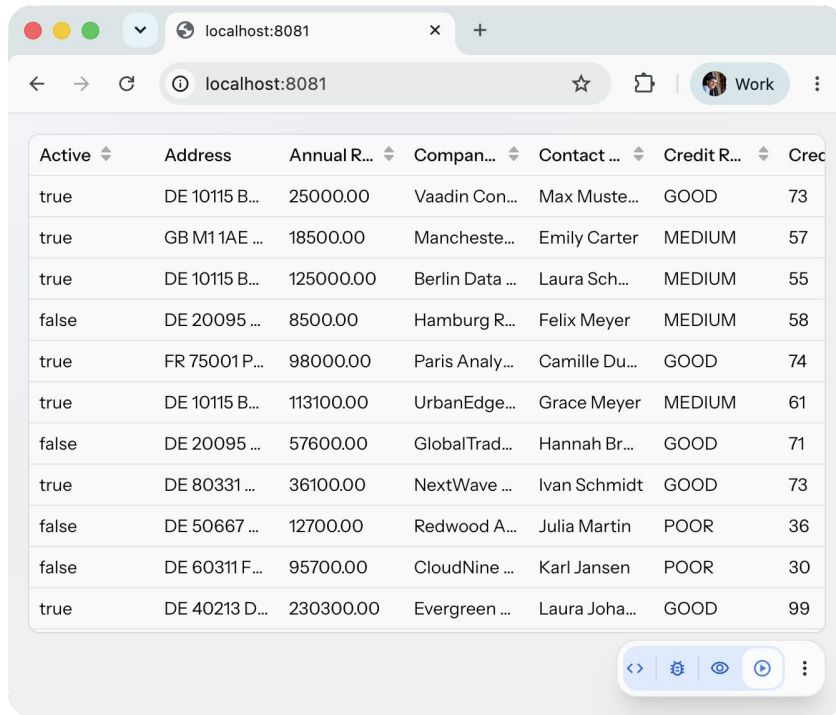
Off-work mostly outdoors;



THREE LINES OF CODE

Load data, show a grid

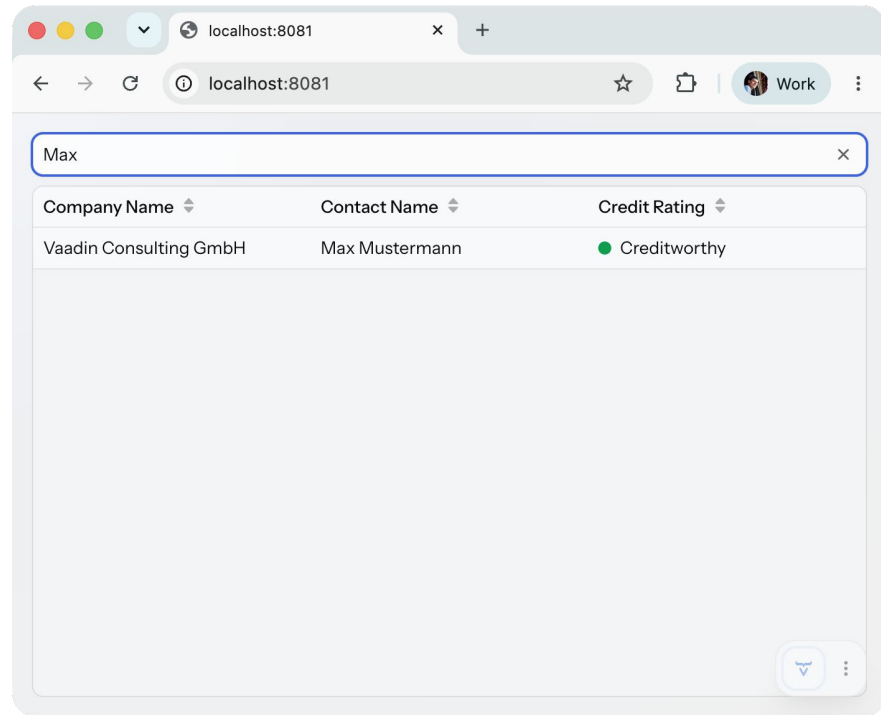
```
var grid = new Grid<>(Customer.class);  
grid.setItems(repository.findAll());  
add(grid);
```



Active	Address	Annual R...	Compan...	Contact ...	Credit R...	Crec
true	DE 10115 B...	25000.00	Vaadin Con...	Max Muste...	GOOD	73
true	GB M11AE ...	18500.00	Mancheste...	Emily Carter	MEDIUM	57
true	DE 10115 B...	125000.00	Berlin Data ...	Laura Sch...	MEDIUM	55
false	DE 20095 ...	8500.00	Hamburg R...	Felix Meyer	MEDIUM	58
true	FR 75001 P...	98000.00	Paris Analy...	Camille Du...	GOOD	74
true	DE 10115 B...	113100.00	UrbanEdge...	Grace Meyer	MEDIUM	61
false	DE 20095 ...	57600.00	GlobalTrad...	Hannah Br...	GOOD	71
true	DE 80331 ...	36100.00	NextWave ...	Ivan Schmidt	GOOD	73
false	DE 50667 ...	12700.00	Redwood A...	Julia Martin	POOR	36
false	DE 60311 F...	95700.00	CloudNine ...	Karl Jansen	POOR	30
true	DE 40213 D...	230300.00	Evergreen ...	Laura Joha...	GOOD	99

Searching with a text field

- Generic OR search across all fields
- Messy, because it finds a lot



Many fields, its own syntax, its own logic

When it has to get precise

Customer Grid – Lazy Filter

Company Name	Contact Name	Credit Rating
Vaadin Consulting GmbH	Max Mustermann	● Creditworthy
Manchester Components Ltd	Emily Carter	● Limited
Berlin Data Works	Laura Schmidt	● Limited
Hamburg Retail Group	Felix Meyer	● Limited
Paris Analytics SAS	Camille Dubois	● Creditworthy
UrbanEdge Software Berlin	Grace Meyer	● Limited
GlobalTrade Partners Hamburg	Hannah Brown	● Creditworthy
NextWave Systems Munich	Ivan Schmidt	● Creditworthy
Redwood Analytics Cologne	Julia Martin	● At risk
CloudNine Services Frankfurt	Karl Jansen	● At risk

Many fields, its own syntax, its own logic

When it has to get precise

The screenshot shows a web browser window with the address bar displaying 'localhost:8081/lazy-form'. The page title is 'Customer Grid - Lazy Form Filter'. The form contains the following fields:

- Company name (text input)
- Contact name (text input)
- Email (text input)
- City (text input)
- Country (dropdown menu)
- Status (dropdown menu, currently set to 'Any')
- Credit rating (dropdown menu)
- Annual revenue (€) (range selector with 'min' and 'max' inputs)
- Customer since (range selector with 'from' and 'until' date inputs)
- Last order (range selector with 'from' and 'until' date inputs)

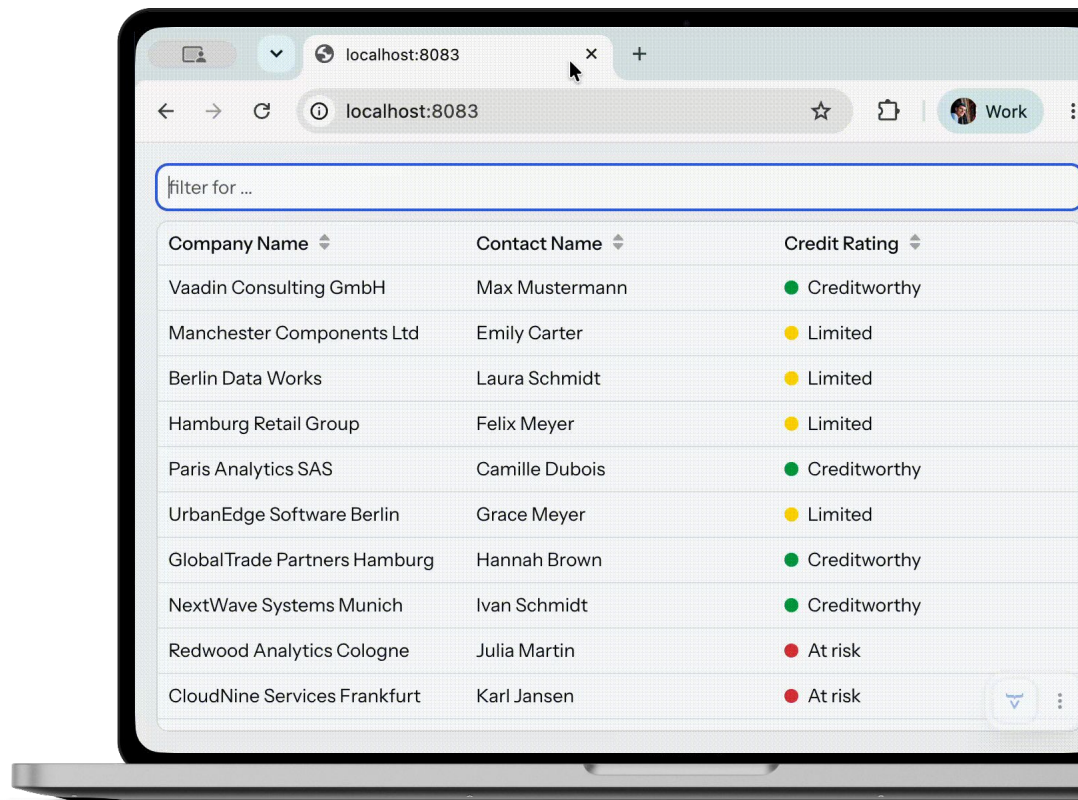
At the bottom right of the form are 'Search' and 'Reset' buttons. Below the form is a table with the following data:

Company Name	Contact Name	Email	Phone	Address	Credit Rating
Vaadin Consulting G...	Max Mustermann	max.mustermann@va...	+493010007919	DE 10115 Berlin - Innovation Way ...	● Creditworthy
Manchester Compon...	Emily Carter	emily.carter@manche...	+441611015838	GB M11AE Manchester - Market ...	● Limited
Berlin Data Works	Laura Schmidt	laura.schmidt@berlin-...	+493010023757	DE 10115 Berlin - Torstrasse 99	● Limited
Hamburg Retail Group	Felix Meyer	felix.meyer@hamburg...	+494010031676	DE 20095 Hamburg - High Stree...	● Limited
Paris Analytics SAS	Camille Dubois	camille.dubois@paris...	+33122222505	FR 75001 Paris - Rue de Drouot 14	● Creditworthy

Fight for Simplicity

Why not use my own words to search?

→ with AI



Talking to a model from Java

What are the options in my JVM?

FRAMEWORKS

LangChain4j · [Spring AI](#) · Koog

Talking to a model from Java

What are the options in my JVM?

FRAMEWORKS

LangChain4j · [Spring AI](#) · Koog

```
// Field Declaration  
private final ChatClient chatClient;
```

THREE APPROACHES

How can I process information in an LLM using Java?

01

Tool Calling

02

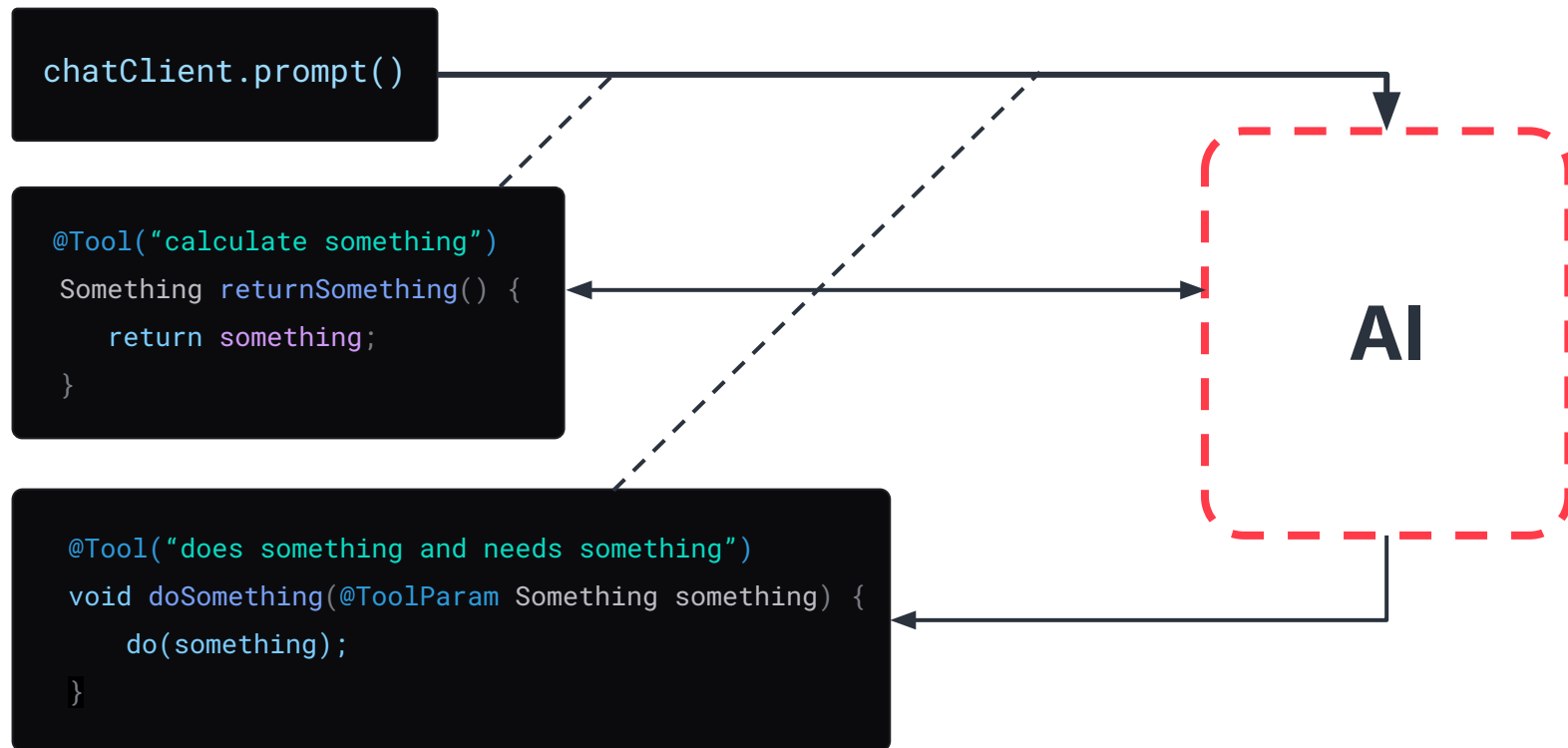
Structured
Output

03

Hybrid

Tool Calling

Brief Introduction



FORWARD ALL THE DATA TO THE AI

Simple Approach

```
chatClient.prompt()  
    .system("Here are my customers: ...")  
    .user("show customers from Paris")  
    .call()  
    .chatResponse();
```

FORWARD ALL THE DATA TO THE AI

Simple Approach

```
chatClient.prompt()  
    .system("Call getAllCustomers...")  
    .user("show customers from Paris")  
    .tools(this)  
    .call()  
    .chatResponse();
```

```
@Tool(description = "Returns all customers")  
List<Customer> getAllCustomers() {  
    return customerRepository.findAll();  
}
```

FORWARD ALL THE DATA TO THE AI

Simple Approach

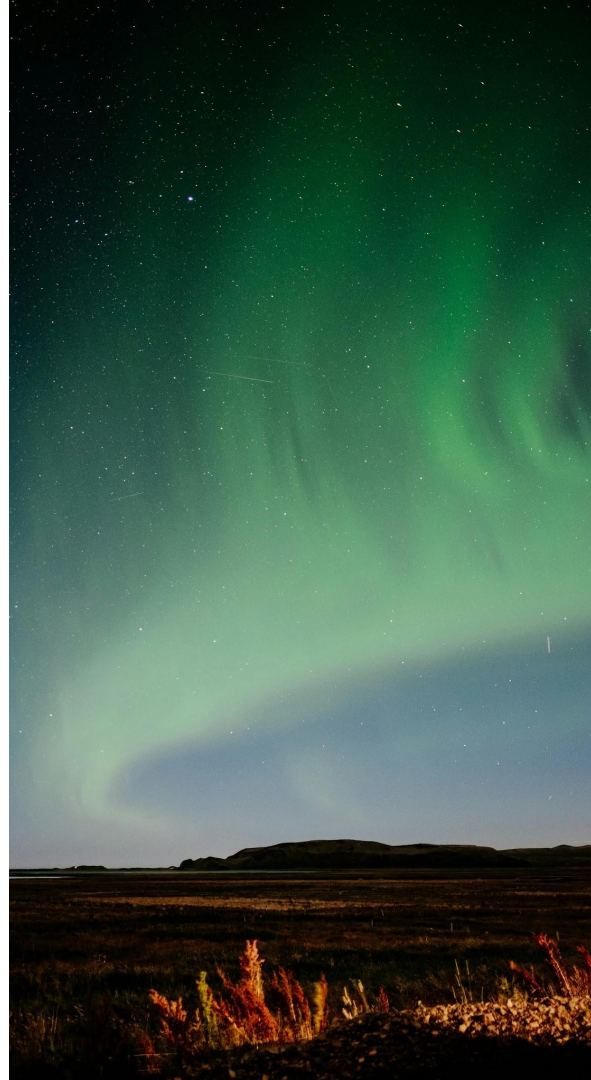
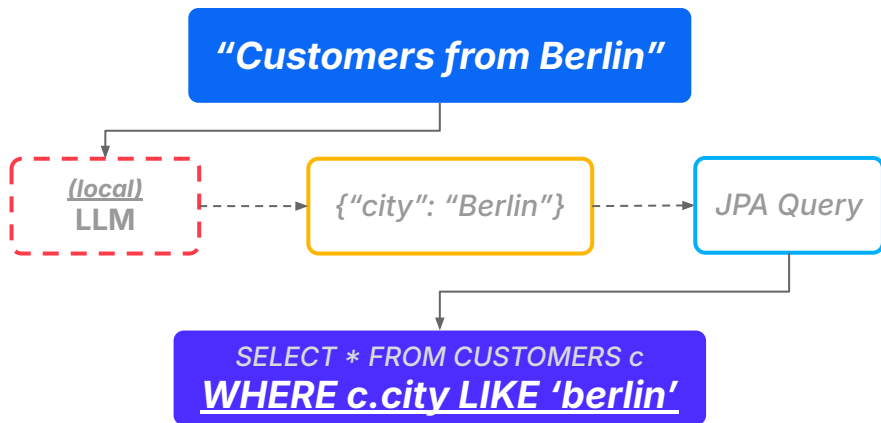
```
chatClient.prompt()  
    .system("Call getAllCustomers...")  
    .user("show customers from Paris")  
    .tools(this)  
    .call()  
    .chatResponse();
```

```
@Tool(description = "Returns all ...")  
List<Customer> getAllCustomers() {  
    return customerRepository.findAll();  
}
```

```
@Tool(description = "Shows the matching ...")  
void showCustomers(  
    @ToolParam(description = "...") Set<Long> ids) {  
    var relevantCustomers = repo.findAllById(ids);  
    grid.setItems(relevantCustomers);  
}
```

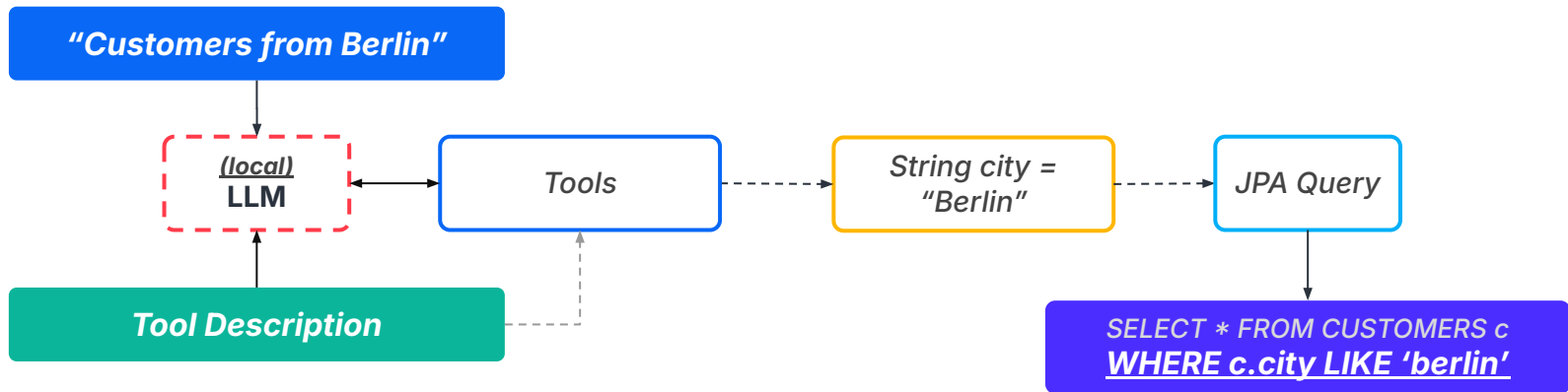

The better way

- *Extract the filter criteria from user prompt, build the query yourself*
- + *only the query exposes, never the data*



Approach 1: Tool Calling

- + *The LLM calls your method*
- + *The parameter description is the contract*
- + *Result end in a Tool call as side effect*



Three parameters, AND-combined, all optional

Approach 1: Target method

```
@Tool(returnDirect = true, description = "Filters the customer grid.  
Every parameter is optional and AND-combined; null ignores one.")  
void searchCustomers(  
    @ToolParam(description = "city, e.g. \"Berlin\"")  
    String city,  
    @ToolParam(description = "last order date, ISO yyyy-MM-dd")  
    LocalDate lastOrderDate,  
    @ToolParam(description = "credit rating: GOOD, MEDIUM or POOR")  
    CreditRating creditRating  
) { ... }
```

Checklist: Exemplary Queries

- 01 *"Customers from Berlin"* ✓
- 02 *"Customers from Berlin or Hamburg"*
- 03 *"All customers except those from Berlin"*
- 04 *"Kunden aus München"* ✓
- 05 *"Customers who ordered yesterday"* ✓
- 06 *"Creditworthy customers from Frankfurt or Cologne who ordered in the last 6 months"*

filtervalue + operator + negate per field

Adding more parameters

```
@Tool(returnDirect = true, description = "Filters the customer ...")
void searchCustomers(
    ...
    @ToolParam(description = "last order date, ISO yyyy-MM-dd")
    LocalDate lastOrderDate,
    @ToolParam(description = "compare the date: EQUALS, ...")
    Operator lastOrderDateOperator,
    @ToolParam(description = "true to exclude the matching ...")
    Boolean lastOrderDateNegate,
    ...
) { ... }
```

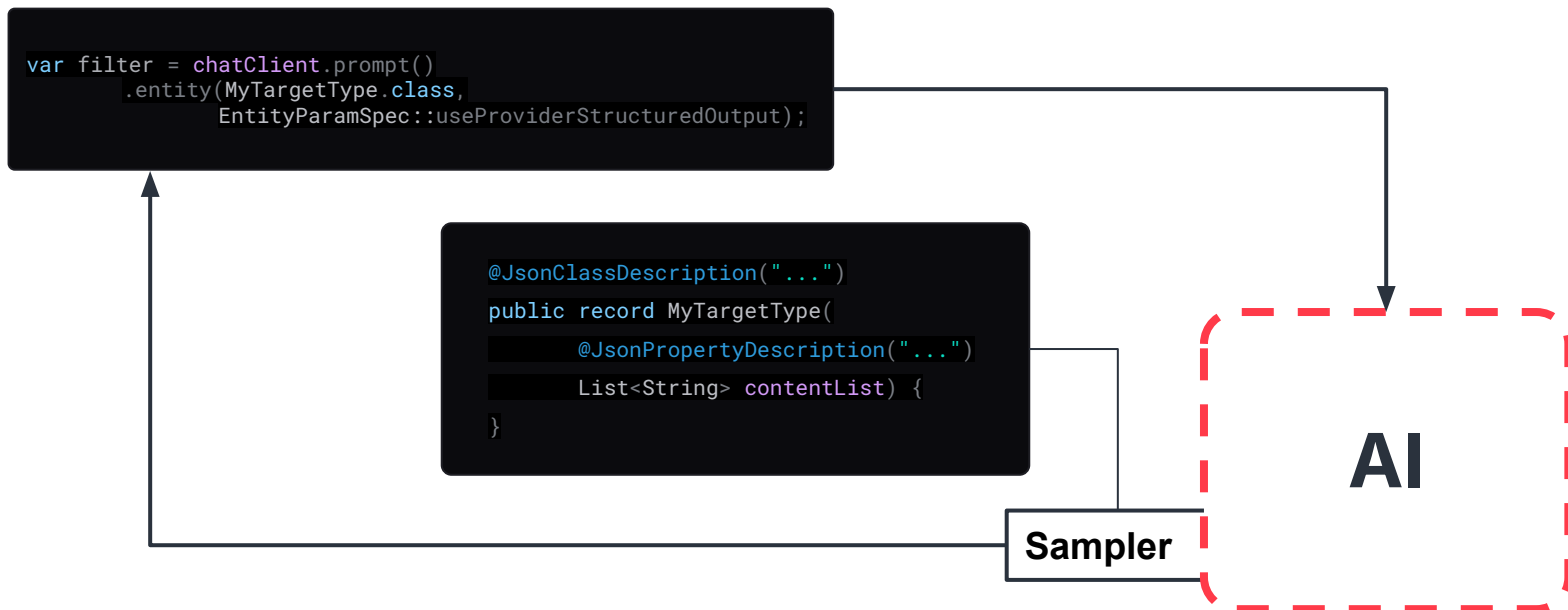
→ *Three parameters become nine parameters*

Checklist: Exemplary Queries

- 01 *"Customers from Berlin"* ✓
- 02 *"Customers from Berlin or Hamburg"*
- 03 *"All customers except those from Berlin"* ✓
- 04 *"Kunden aus München"* ✓
- 05 *"Customers who ordered yesterday"* ✓
- 06 *"Creditworthy customers from Frankfurt or Cologne who ordered in the last 6 months"* ⚠

Structured Output

Brief Introduction

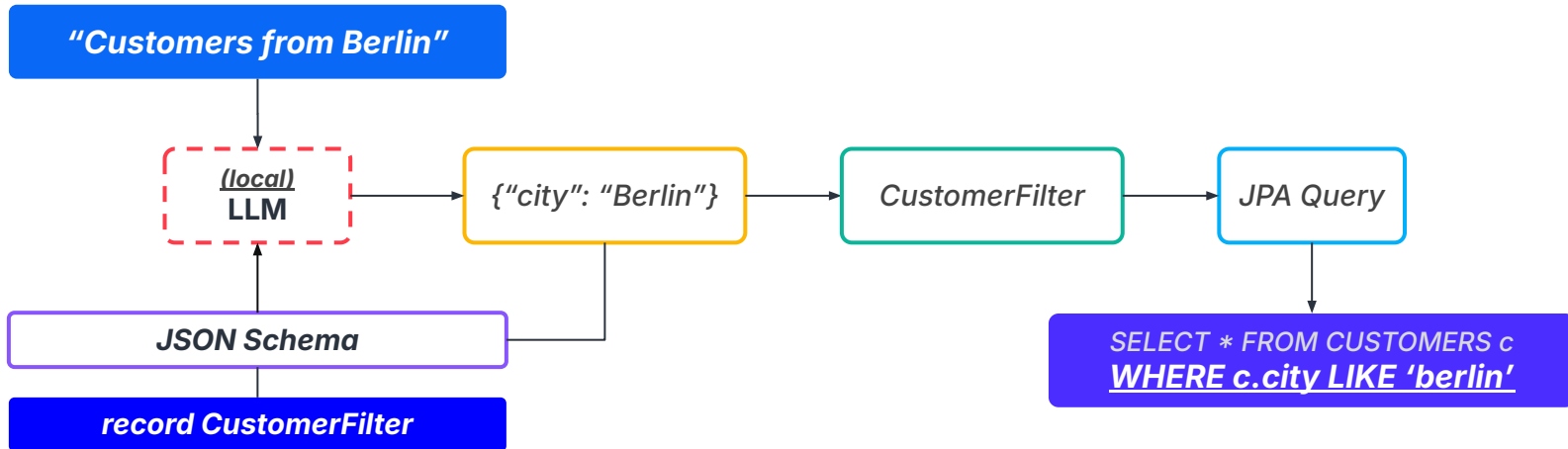


From method signature to data structure

Approach 2: Structured Output

→ *Define a structure instead of listening parameters*

+ *One call, one typed result*



`.entity()` instead of `.chatResponse()`

Approach 2: One call, one typed result

```
var filter = chatClient.prompt()  
    .system(systemPrompt(LocalDate.now()))  
    .user(naturalLanguageQuery)  
    .call()  
    .entity(CustomerFilter.class,  
        EntityParamSpec::useProviderStructuredOutput);
```

Description lives in the annotations

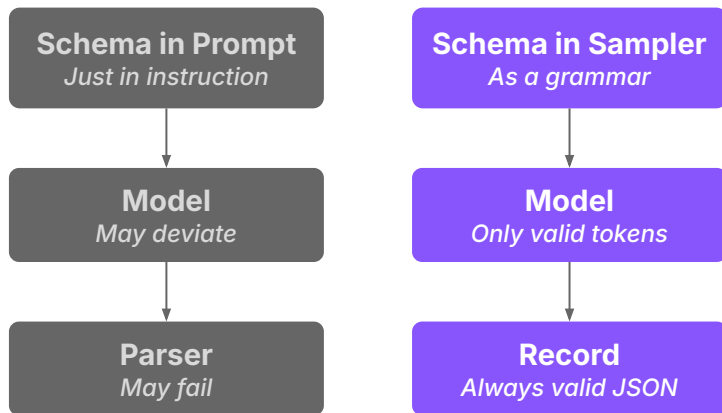
Approach 2: Target Structure

```
public record CustomerFilter(  
    @JsonPropertyDescription("all conditions ...")  
    List<Condition> conditions) {}  
  
@JsonClassDescription("One condition on one field; ...")  
public record Condition(  
    String field,  
    Operator operator,  
    List<String> values,  
    boolean negate) {  
  
    public enum Operator { CONTAINS, EQUALS,  
        GREATER_OR_EQUAL, LESS_OR_EQUAL, ...}  
  
}
```

What that guarantees – and what it rules out

The schema is a constraint, not a request

- Schema goes to Ollama as format, returns well formed JSON, no retry
- Good for small models — the result format costs no tokens
- Since the result form is defined, the model can focus more on the content
- A grammar and a tool call compete for the same output slot



Checklist: Exemplary Queries

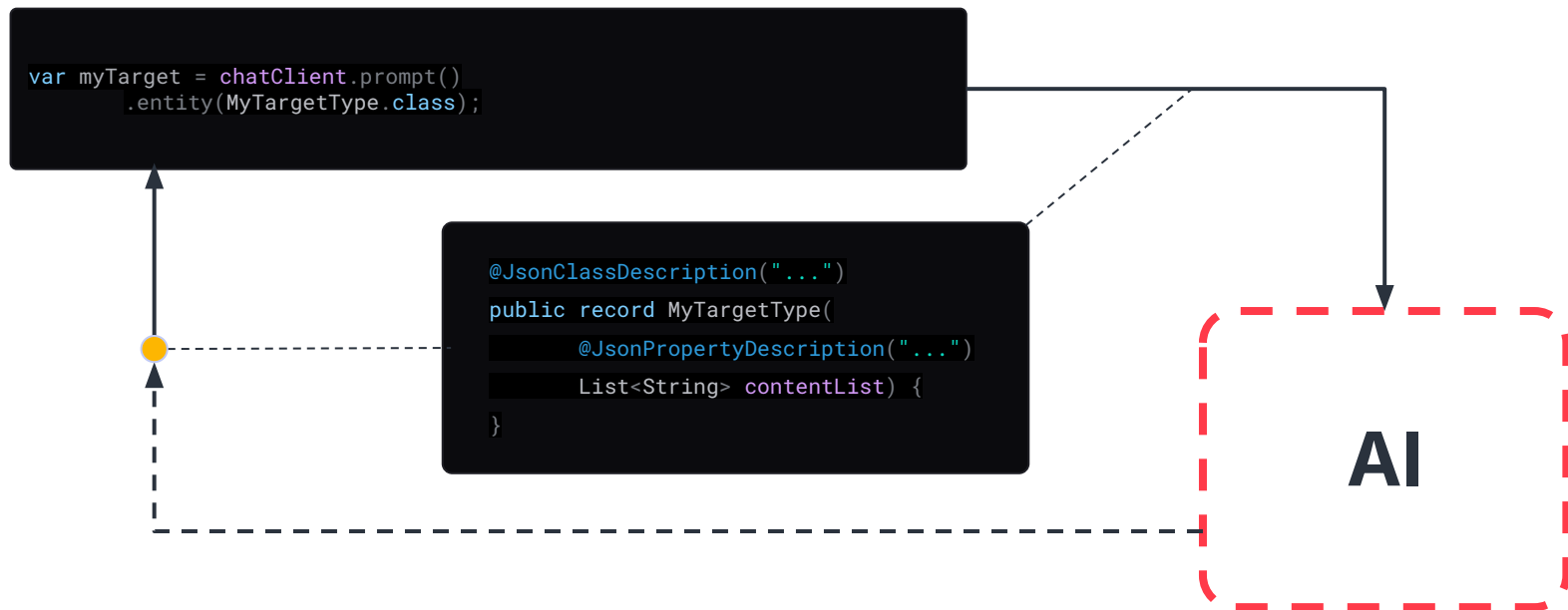
- 01 *"Customers from Berlin"* ✓
- 02 *"Customers from Berlin or Hamburg"* ✓
- 03 *"All customers except those from Berlin"* ✓
- 04 *"Kunden aus München"* ✓
- 05 *"Customers who ordered yesterday"* ✓
- 06 *"Creditworthy customers from Frankfurt or Cologne who ordered in the last 6 months"* ✓

Tool Calling + Structured Output

Hybrid

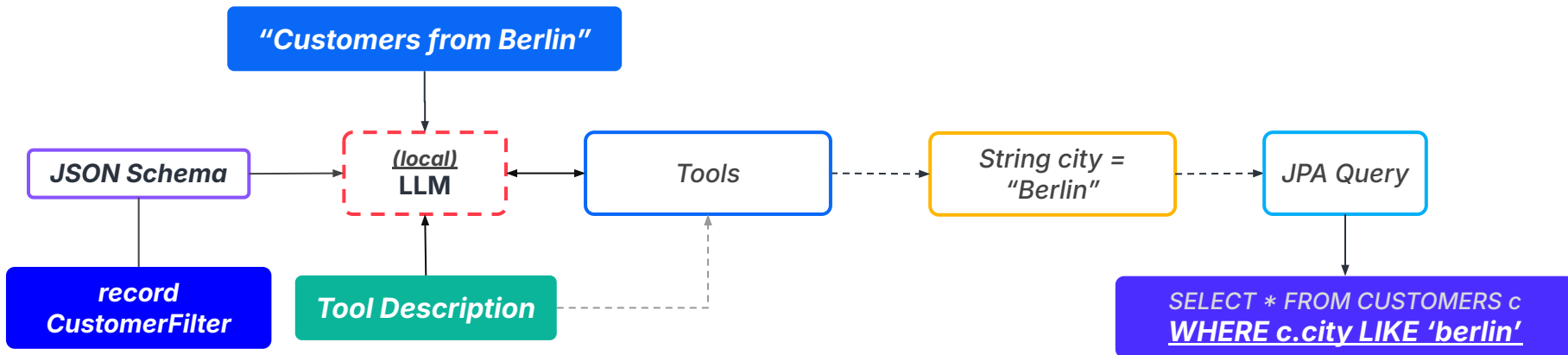
Hybrid

Brief Introduction



Approach 3: Hybrid

- *Target structure plus tools for context*
- + *Structure as in approach 2, context on demand*



Approach 3: Hybrid

```
CustomerFilter requestFilter(String naturalLanguageQuery) {  
    return chatClient.prompt()  
        .system(SYSTEM_PROMPT)  
        .user(naturalLanguageQuery)  
        .tools(this)  
        .call()  
        .entity(CustomerFilter.class);  
}  
  
@Tool(description = "Current date and time")  
LocalDateTime currentLocalDateTime() {  
    return LocalDateTime.now();  
}
```

Checklist: Exemplary Queries

- 01 *"Customers from Berlin"* ✓
- 02 *"Customers from Berlin or Hamburg"* ✓
- 03 *"All customers except those from Berlin"* ✓
- 04 *"Kunden aus München"* ✓
- 05 *"Customers who ordered yesterday"* ✓
- 06 *"Creditworthy customers from Frankfurt or Cologne who ordered in the last 6 months"* ✓

Add every detail to the Prompts

01

Time Specificity

"Yesterday" is a specific point in time, not a range.

*a single relative DAY ("yesterday", "today") is ONE exact day: **EQUALS** that day, never "from that day on"*

02

Toponym Translation

Toponyms must be translated (e.g., "München" to English).

***City names are stored in English** - Berlin, Munich, Cologne, Dusseldorf - so translate a German one before passing it*

03

Tool Rules

Enforce tool rules: Only call tools when strictly necessary to save tokens.

*You have one tool, `currentLocalDateTime`. **Use it ONLY** when the request names a date relative to today (yesterday) ...*

Natural language in, an ordinary AssertJ assertion out

Testing a prompt like any other method

```
@Test
void findsCustomersInOneGermanWrittenCity() {
    assertThat(search("show me all customers in München"))
        .extracting(Customer::getId)
        .containsExactlyInAnyOrderElementsOf(
            expectedIds(customer -> city(customer).equals("Munich")));
}
```

Benchmark Setup & Coverage

BENCHMARK SETUP

```
// MODEL
qwen3:8b via Ollama

// HARDWARE
MacBook M2 Pro 32gb

// PARAMETERS
Temperature 0.0
num-ctx 4096
Thinking Mode false
```

6

Cases

Robustness cases

whether the model reliably hits a filter that already exists

9

CASES

Capability cases

what the filter type can express

Benchmark queries

QUERY	Tool Flat	Tool ext.	Str. Outp.	Hybrid
show me all customers in Berlin	✓ PASS	✓ PASS	✓ PASS	✓ PASS
show me customers from Berlin or Hamburg	✗ n/a	✗ n/a	✓ PASS	✓ PASS
show me all customers except from Berlin	✗ n/a	✓ PASS	✓ PASS	✓ PASS
creditworthy customers in Hamburg	✓ PASS	✓ PASS	✓ PASS	✓ PASS
...placed an order in the last 12 months	✗ n/a	✗ n/a	✓ PASS	✓ PASS
...placed an order in the last week	✗ n/a	✗ n/a	✓ PASS	✓ PASS
...last ordered between 2024-07-01 and 2025-03-31	✗ n/a	✗ n/a	✓ PASS	✓ PASS
Kunden, die zuletzt am 18.11.2025 bestellt haben	✓ PASS	✓ PASS	✓ PASS	✓ PASS
show me all customers who are not creditworthy	✗ n/a	✓ PASS	✓ PASS	✓ PASS
Sum	3 / 9	5 / 9	9 / 9	9 / 9

Benchmark queries

QUERY	Tool Flat	Tool ext.	Str. Outp.	Hybrid
show me all customers in München	✓ PASS	✓ PASS	✓ PASS	✓ PASS
zeige mir alle Kunden aus Köln	✓ PASS	✓ PASS	✓ PASS	✓ PASS
show all customer who ordered yesterday	✓ PASS	✓ PASS	✓ PASS	✓ PASS
zeige mir alle Kunden die gestern was bestellt haben	✓ PASS	✓ PASS	✓ PASS	✓ PASS
Nice weather today, isn't it?	✓ PASS	✓ PASS	✓ PASS	✓ PASS
show me all customers	✓ PASS	✓ PASS	✓ PASS	✓ PASS
Sum	6 / 6	6 / 6	6 / 6	6 / 6

Only the queries every approach can answer

What expressiveness costs*

Approach	Req/Prompt	Tokens/Prompt	ms/Request	Median ms/Prompt	Spread	Range
Tool Flat	1,22	663	1.005 ms	1.228 ms	1.210-1.341	±5 %
Tool ext.	1,22	1.694	1.619 ms	1.978 ms	1.953-2.665	±18 %
Structured Output	1,00	803	1.376 ms	1.376 ms	1.366-1.619	±9 %
Hybrid	1,22	2.020	2.468 ms	3.016 ms	2.968-3.419	±7 %

* 9 queries · 10 runs · qwen3:8b — the subset all four approaches can express

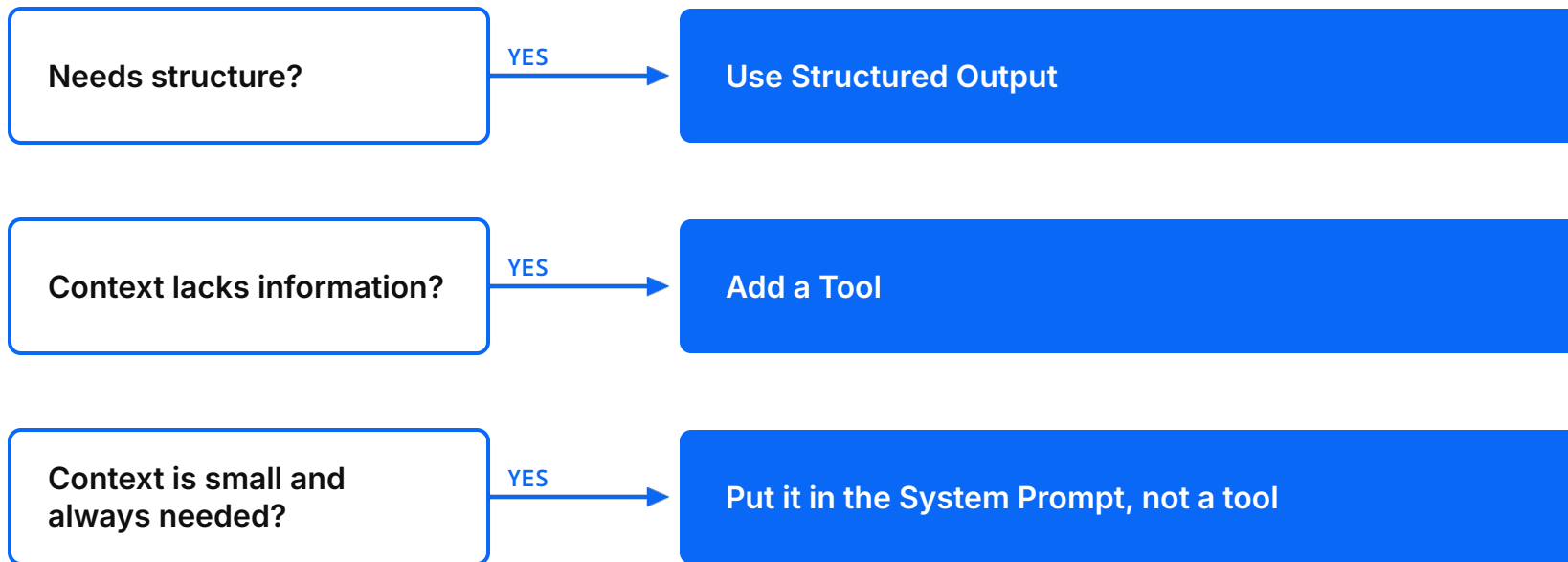
Different query counts – that is the result, not a flaw

Cost of each approach

Approach	Tests	Req./Anfr.	Tok./Anfr.	ms/Req.	Median ms	Range	Spread
Tool flat	9	1,22	662	990 ms	1.210 ms	1.201–1.320	±5 %
Tool ext.	11	1,18	1.633	1.643 ms	1.942 ms	1.799–2.216	±11 %
Structured Output	15	1,00	812	1.588 ms	1.588 ms	1.563–1.721	±5 %
Hybrid	15	1,27	2.103	2.594 ms	3.286 ms	3.191–3.458	±4 %

10 runs · qwen3:8b · each approach on every query it can express

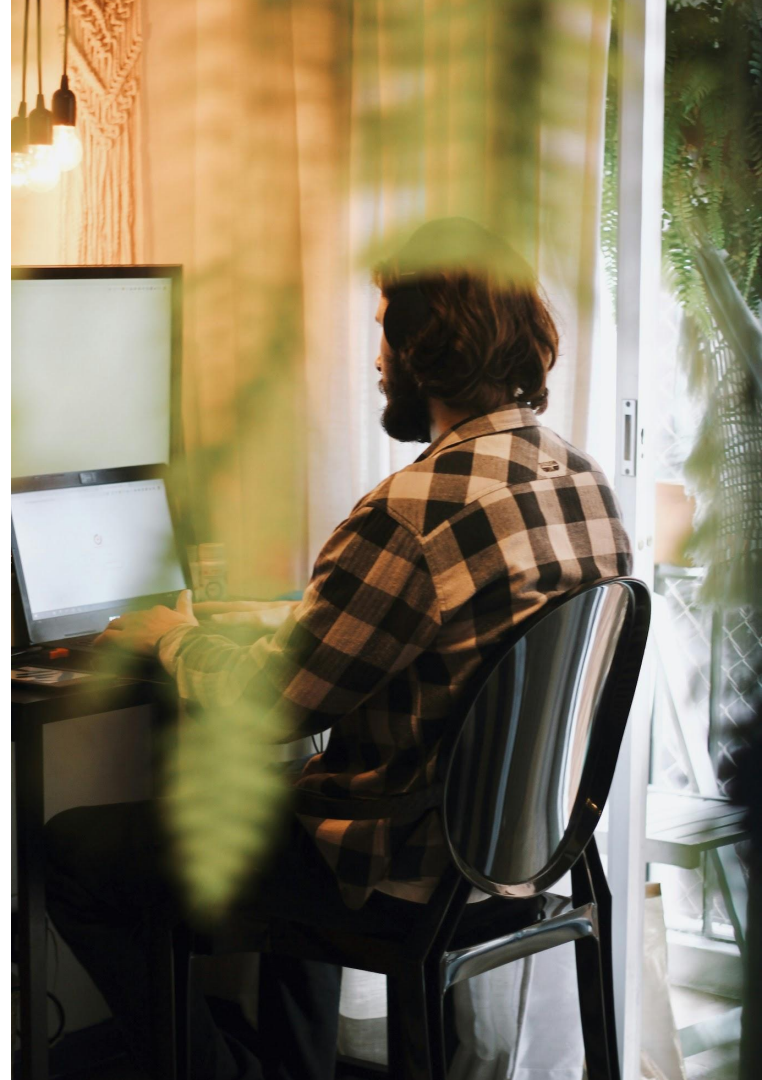
Architectural Decision Tree



ADDITIONAL INFORMATION

What justifies using a tool?

- Everything what is too big for a prompt and what changes constantly
- Dynamic context (like datetime)
- external Information (weather, current prices, business data)
- deterministic Businesslogic (valid ranges creditworthy)



**What about
other local
models?**

FOUR REASONS

Why local at all?

01

Independent of
conference
Wi-Fi

02

No data leaves
the building

03

No cost per
token

04

Predictable
latency

Which model?

Same test approach, additional testcases, different models

Model	Tool Flat 14 Cases	Tool ext. 18 Cases	Str. Outp. 21 Cases	Hybrid 21 Cases	Total	RAM
qwen3:8b	70 (100 %)	90 (100 %)	105 (100 %)	105 (100 %)	370/370	5,5 GB
qwen3.5:4b-mlx	70 (100 %)	90 (100 %)	100 (95 %)	105 (100 %)	365/370	3,9 GB
gemma4:26b-mlx	70 (100 %)	75 (83 %)	100 (95 %)	100 (95 %)	345/370	16,3 GB
qwen3.5:9b-mlx	52 (74 %)	85 (94 %)	96 (91 %)	105 (100 %)	338/370	9,6 GB
llama3.1:8b	65 (93 %)	40 (44 %)	95 (90 %)	95 (90 %)	295/370	4,9 GB

5 runs per case · 370 executions per model · MacBook M2 Pro · Ollama 0.33.3

Which approach?

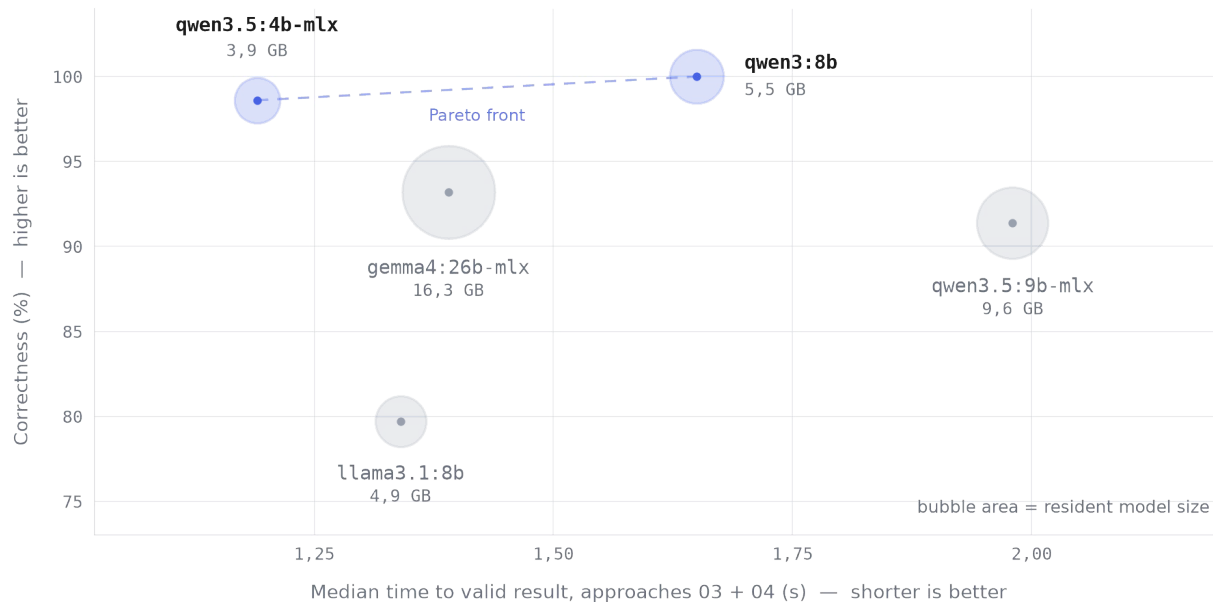
Same speed, more correct — at 1.604 → 2.100 tokens per request

Model	Str. Outp. Correct	Str. Outp. p50	Str. Outp. p95	Hybrid Correct	Hybrid p50	Hybrid p95
qwen3:8b	105/105	1,6 s	3,3 s	105/105	1,7 s	3,1 s
qwen3.5:4b-mlx	100/105	1,2 s	2,6 s	105/105	1,2 s	2,3 s
gemma4:26b-mlx	100/105	1,7 s	3,7 s	100/105	1,1 s	2,8 s
qwen3.5:9b-mlx	96/105	2,0 s	4,4 s	105/105	2,0 s	3,7 s
llama3.1:8b	95/105	1,2 s	2,8 s	95/105	1,5 s	2,6 s

5 runs per case · 21 cases per approach · MacBook M2 Pro · Ollama 0.33.3

Small beats big

Correctness, speed and footprint in one picture



Correctness over all approaches · median over 03 + 04 · MacBook M2 Pro

Conclusions

- 01 Target structure beats model choice
- 02 Structured Output is the default but not a must
- 03 Tool Calling is for context and potential usage of side effects
- 04 Token cost comes from roundtrips
- 05 4B + 8B models solve this reliably

Where to go from here?

Get started with Vaadin



Slides
Demo Project

seb@vaadin.com

[github.com/
SebastianKuehnau/
vaadin-grid-ai-filter/
tree/demo](https://github.com/SebastianKuehnau/vaadin-grid-ai-filter/tree/demo)

Meet us at
Vaadin Create 2026
in Barcelona
Oct 27-28, 2026

Discount Code:
JUGDortmund199

[vaadin.com/
vaadin-create-2026](https://vaadin.com/vaadin-create-2026) vaadin}>